

YABA COLLEGE OF TECHNOLOGY

SCHOOL OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

**DESIGN AND IMPLEMENTATION OF A DIGITAL PROJECT SUPERVISION
AND PROGRESS TRACKING SYSTEM USING A RULE-BASED MILESTONE
PROGRESS TRACKING ALGORITHM (A CASE STUDY OF YABA COLLEGE
OF TECHNOLOGY)**

BY

[STUDENT FULL NAME]

MATRICULATION NUMBER: [MATRIC NUMBER]

**BEING A PROJECT SUBMITTED TO THE DEPARTMENT OF COMPUTER
SCIENCE, SCHOOL OF TECHNOLOGY, YABA COLLEGE OF
TECHNOLOGY, IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR
THE AWARD OF HIGHER NATIONAL DIPLOMA (HND) IN COMPUTER
SCIENCE**

OCTOBER 2026

TABLE OF CONTENTS

CHAPTER ONE	3
1.1 Background of the Study	3
1.2 Statement of the Problem	5
1.3 Aim of the Study	5
1.4 Objectives of the Study	5
1.5 Research Questions & Hypothesis	6
1.6 Significance / Justification	6
1.7 Scope of the Study	7
1.8 Methodology Overview	7
1.9 Limitations of the Study	8
1.10 Definition of Terms	8
CHAPTER TWO	10
2.0 Introduction	10
2.1 Conceptual Framework	10
2.2 Theoretical Framework	11
2.3 Empirical Framework	11
2.4 Review of Related Work	12
2.5 Gap in Literature	21
CHAPTER THREE	22
3.0 Introduction	22
3.1 System Development Methodology	22
3.2 Analysis of the Existing System	23
3.3 Analysis of the Proposed System	24
3.4 System Requirements	25
3.5 System Architecture	26
3.6 Database and Data Design	27
3.7 Rule-based Milestone Progress and Project Health Algorithms	29
3.8 Security and Access Control	32
3.9 Development and Deployment Tools	33
CHAPTER FOUR	34
4.0 Introduction	34
4.1 Implementation Overview	34
4.2 User Interface Implementation	35
4.3 Test Design and Acceptance Criteria	39
4.4 Implementation and System Behaviour	42
4.5 User Guide	45
4.6 Chapter Summary	46
REFERENCES	47

CHAPTER ONE

INTRODUCTION

1.1 Background of the Study

Project supervision is the process by which an academic supervisor guides a student through the stages of conceiving, developing and documenting a final year or Higher National Diploma (HND) project. It is one of the most demanding responsibilities placed on academic staff, since a single supervisor may be required to guide dozens of students at once, while each student progresses through the same broad stages of proposal writing, literature review, design, implementation and testing at a different pace. In most Nigerian polytechnics, this process is still conducted almost entirely on paper: students record supervision meetings in handwritten logbooks, corrections are made directly on printed chapters, and appointments are scheduled informally by knocking on a supervisor's office door or sending a text message.

This manual arrangement creates several recurring difficulties. Logbooks are easily misplaced by students between semesters, and a supervisor who is away from the department for even a short period has no way of knowing how far a student has progressed since the last physical meeting. Feedback on a submitted chapter can take days or weeks to reach a student, not because the supervisor is unwilling to help, but because there is no channel through which comments can be delivered outside a face-to-face encounter. Where a department runs several hundred HND projects in a single session, the Head of Department and the project coordinator have no consolidated view of which students are on schedule, which have stalled, and which supervisors are overloaded; this information exists only in scattered, unsynchronised paper records held by individual supervisors.

Yaba College of Technology, popularly known as YABATECH, is a federal tertiary institution located in Yaba, Lagos State, and is widely regarded as the first higher educational institution established in Nigeria. Its roots lie in Yaba Higher College, which was founded in 1932 to provide tertiary education, mostly in vocational and teaching subjects. Following the recommendation of the Elliot Commission on higher education in West Africa, the Higher College was closed and its students were moved in 1948 to form the nucleus of the University College, Ibadan, while the Yaba Technical Institute, which

started day and evening classes in October 1947, took over its activities and premises (Yaba College of Technology, n.d.).

The Institute was later renamed Yaba College of Technology, and in 1969 a Federal Government decree established it as an autonomous institution. Under the Federal Polytechnics Act of 1979, it was renamed Federal Polytechnic, Yaba, but it reverted to its original name in 1980; until 1979 it was the only institution in Nigeria offering Higher Diploma courses. Today the College operates under the regulation of the National Board for Technical Education (NBTE) and offers National Diploma (ND), Higher National Diploma (HND) and Post-HND programmes in science, technology, commerce and related fields. It was also the first higher institution in Nigeria to establish a full-fledged Centre for Entrepreneurship Development (Yaba College of Technology, n.d.). The Department of Computer Science, whose HND project supervision is the subject of this study, operates within this institutional setting, which makes the College a representative and appropriate case study for examining the challenges of project supervision in Nigerian polytechnics.

At Yaba College of Technology, these problems are visible every session in the Department of Computer Science. Final-year HND students are required to complete a project as a condition for graduation, and the department assigns supervisors from a relatively small pool of academic staff to a much larger number of students. Supervision meetings are held physically, in the supervisor's office, and the only record of what was discussed, and of the corrections made, is whatever the student manages to write in a personal notebook. When a student wishes to confirm the date of the last correction made to a chapter, or a supervisor wishes to review how many students have completed their literature review, no digital record exists to consult. This lack of a structured, time-stamped record of supervision activity makes it difficult to resolve disagreements about whether feedback was given, and makes it almost impossible for departmental administration to track overall project progress across cohorts.

Advances in web-based information systems make it possible to supplement paper-based supervision with a shared digital platform accessible from an internet-connected device. The platform can keep timestamped topic decisions, submission records, chapter reviews and feedback, while applying explicit rules to represent project progress. In the implementation developed for this study, the five chapter stages have equal weights of 20 per cent each; a stage contributes its weight only when the assigned supervisor approves

the current chapter submission. Topic approval is the entry condition for chapter work, while final project approval is recorded separately after every chapter is approved. A further rule set classifies project health using activity, submission recency and the scheduled defence date. Together, these measures provide a transparent basis for monitoring supervision in the Department of Computer Science, Yaba College of Technology.

1.2 Statement of the Problem

The Department of Computer Science at Yaba College of Technology currently has no digital system for supervising and tracking HND research projects, so supervision records are kept on paper by individual supervisors and students, feedback on submitted work is delayed until the next physical meeting, and neither the project coordinator nor the Head of Department has a reliable, up-to-date view of how far any given student has progressed. This study therefore designs and implements a digital project supervision and progress tracking system, driven by a rule-based milestone progress tracking algorithm, to replace this manual arrangement with a structured, real-time record of supervisor-student communication and project progress.

1.3 Aim of the Study

The aim of this study is to design and develop a digital project supervision and progress tracking system that applies a rule-based milestone progress tracking algorithm to automate the monitoring of student project progress and streamline supervisor-student communication at Yaba College of Technology.

1.4 Objectives of the Study

To achieve the stated aim, the study pursues the following specific objectives:

- i. To investigate the existing manual project supervision process in the Department of Computer Science, Yaba College of Technology, and identify its specific shortcomings.
- ii. To design the system architecture, database structure and a rule-based milestone progress tracking algorithm capable of computing a student's project progress from defined supervision milestones.

- iii. To develop a web-based platform that implements the designed architecture and algorithm, incorporating separate interfaces for students, supervisors and the Head of Department for departmental oversight.
- iv. To implement milestone weighting, automatic progress-percentage computation, explicit topic and final-approval conditions, and event notifications together with rule-based project-health indicators.
- v. To verify the developed system against its functional objectives using black-box acceptance cases and automated regression tests, and to define an evaluation procedure for subsequent student and supervisor feedback.

1.5 Research Questions & Hypothesis

The following research questions guided the study:

- i. What are the specific shortcomings of the existing manual project supervision process in the Department of Computer Science, Yaba College of Technology?
- ii. What milestones, rules and weighting scheme should a rule-based algorithm use to compute an accurate and meaningful progress percentage for an HND project?
- iii. What features must a digital project supervision and progress tracking system provide to meet the needs of students, supervisors and departmental project oversight at Yaba College of Technology?
- iv. How does the developed system, evaluated against the existing manual process, affect the timeliness of supervisor feedback and the visibility of student progress?

Hypothesis: The implementation of a digital project supervision and progress tracking system driven by a rule-based milestone progress tracking algorithm will significantly reduce the average time taken for a supervisor's feedback to reach a student, and will improve the accuracy with which project progress can be reported, compared with the existing manual process.

1.6 Significance / Justification

This study is significant to several groups of stakeholders within Yaba College of Technology. Students benefit from one platform for selecting a supervisor in their department, proposing a topic, working with registered project partners, submitting

chapters and receiving feedback. Chapter states and the progress percentage make approved and outstanding work visible, while activity records explain topic, review and extension decisions. Supervisors benefit from a consolidated view of their projects, matriculation-number identification of students, document review and grading controls, and rule-based indicators of inactivity or deadline pressure. The Head of Department benefits from department-wide project and supervisor summaries, which support follow-up and workload discussions. The institution gains a timestamped record that can support review of the supervision process. The study also provides a documented example of transparent progress rules in a Nigerian polytechnic setting. These are intended benefits; measured reductions in feedback time or improvements in academic outcomes require a separate participant evaluation.

1.7 Scope of the Study

This study focuses on a web-based digital project supervision and progress tracking system for HND students in the Department of Computer Science, Yaba College of Technology, with the 2024/2025 academic session retained as the reference case. The implemented workflow covers registration, student matriculation-number login, departmental supervisor selection, topic submission and review, project partners, chapter submission and revision, feedback, grades, activity records, deadline-extension requests, final approval and defence-date tracking. Five equally weighted chapter milestones provide the progress percentage, while separate activity and schedule rules produce project-health indicators. The Head of Department interface supports departmental monitoring. The application also accepts ND matriculation formats and provides other department options; the report's case-study focus remains Computer Science HND supervision. Plagiarism detection, payment processing, external-examiner allocation, defence-panel management, defence results and certification are outside the implemented scope.

1.8 Methodology Overview

The project combined analysis of the supervision workflow with system design, implementation and functional verification. A sequential development lifecycle organised the initial work into requirements analysis, design, implementation, testing and deployment; later requirement corrections led to targeted revisions. The application uses Next.js 14 with TypeScript, Firebase Authentication and Cloud Firestore for identity and

structured data, private Supabase Storage for submitted files, and Vercel for hosting and scheduled health refresh. Automated handler and session tests provide repeatable checks, while role-based black-box cases define the remaining live acceptance exercise. Chapters Three and Four distinguish implemented behaviour, verified results and evaluation that remains to be completed.

1.9 Limitations of the Study

- i. Access to some existing supervision and project records at Yaba College of Technology was restricted for confidentiality reasons, which limited the depth of analysis of past project outcomes that could be carried out during the existing-system study.
- ii. The study was limited to the Department of Computer Science at a single institution, so the milestone weighting scheme and findings may not be fully generalisable to other departments or polytechnics with different project structures.
- iii. Internet connectivity and the availability of Firebase, Supabase and Vercel affect access to live updates and submission transfer. Authenticated end-to-end operation also depends on the matching service configuration and deployed database rules and indexes.
- iv. The available evidence consists of implementation inspection, automated regression tests and public-page browser checks. A longitudinal study with a real student cohort and a statistically controlled comparison of feedback turnaround time have not been completed; the hypothesis remains to be evaluated.

1.10 Definition of Terms

Digital Project Supervision: The use of a web-based information system, rather than paper records and unscheduled physical meetings, to manage the interaction between a student and an academic supervisor throughout the lifecycle of an HND project.

Progress Tracking: Recording the state of project work and computing its completion percentage from approved chapter milestones; the percentage is distinct from project-health classification and final academic approval.

Milestone: A defined, verifiable project stage with a completion condition and a weight. In this implementation, each of the five chapter stages has a weight of 20 per cent and is complete for progress calculation when its current submission is approved.

Rule-Based Algorithm: An algorithm that applies explicit conditions to stored data. Here, chapter approval conditions produce a progress percentage, and a separate ordered rule set uses activity and defence-related signals to classify project health.

Supervisor: An academic staff user associated with a project in the student's department, responsible for topic decisions, submission review, grading, chapter approval, extension decisions and final approval.

Supervisee: A student linked to a supervisor through a project, either as the lead student or as a registered project partner.

Dashboard: A role-specific screen summarising a user's project records, progress, review workload, notifications or department-level activity.

Notification / Alert: An in-application record of a relevant event, such as partner addition, a topic decision, a submission, feedback or an extension decision. Health badges and reasons indicate schedule or inactivity concerns; email, SMS and push delivery are not part of the implemented notification channel.

CHAPTER TWO

LITERATURE REVIEW

2.0 Introduction

Digital supervision and progress tracking systems for academic projects have attracted growing research attention over the past decade, driven by the recognition that manual, paper-based supervision does not scale well as student enrolment rises and as institutions face increasing pressure to demonstrate the quality and consistency of their project supervision. A parallel body of work has examined rule-based algorithms, expert systems and, more recently, large language model-assisted approaches for automating academic monitoring, feedback generation and progress tracking in educational settings. This chapter reviews the conceptual, theoretical and empirical foundations relevant to a rule-based milestone progress tracking algorithm, critically examines a selection of related published studies, and identifies the specific gap that the present study addresses within the Nigerian polytechnic context.

2.1 Conceptual Framework

The conceptual framework organises the system into Input, System Processing and Output. Inputs include account and department details, student matriculation numbers, the selected supervisor, topic and project-partner records, chapter files, feedback, review decisions, ticket activity, deadlines and a defence date. Processing first verifies the user's role and project access, validates the action, and updates the relevant records. The milestone component counts approved chapter stages and multiplies the count by the fixed 20 per cent chapter weight. Approval of the topic permits chapter work, and approval of all five chapters permits a final submission; the supervisor's final decision is stored separately. A health component evaluates inactivity, submission recency, ticket cadence and defence timing. Outputs are role-specific dashboards, the stage-by-stage breakdown, submission feedback and grades, topic and extension states, activity history, recipient notifications and department summaries. Each new supervision event changes the stored inputs and can therefore change the next displayed output. The framework preserves the supervisor's academic judgement: the rules summarise recorded decisions and activity rather than automatically judging the quality of a report.

2.2 Theoretical Framework

This study is anchored on the Technology Acceptance Model (TAM), proposed by Davis (1989). TAM holds that a user's willingness to adopt and continue using a new information system is shaped primarily by two beliefs: perceived usefulness, which is the degree to which the user believes the system will improve their performance of a task, and perceived ease of use, which is the degree to which the user believes the system requires little effort to operate. Applied to this study, TAM provides the lens through which the eventual adoption of the digital project supervision and progress tracking system by supervisors and students at Yaba College of Technology can be evaluated: a supervisor is more likely to abandon paper logbooks in favour of the platform if it is perceived as genuinely reducing the effort involved in tracking multiple students, and a student is more likely to use the system consistently if the interface for submitting work and viewing progress is perceived as simple and intuitive. Abubakar and Muhammed (2022) applied TAM in a Nigerian tertiary institution to examine the acceptance of internet-based tools for academic purposes and found that perceived usefulness and perceived ease of use remained the dominant predictors of actual system use among Nigerian students and staff, a finding that supports the relevance of TAM to the present study and informs the emphasis placed on a simple, low-friction interface in the design of the proposed system.

2.3 Empirical Framework

A number of empirical studies provide evidence that digital supervision and progress-feedback systems produce measurable improvements over manual processes. van der Merwe, du Toit and Kruger (2020) implemented a model-based academic progress feedback framework at a South African university and, evaluating it through a user survey, found that the framework was readily accepted by staff and students and conformed to the feature requirements identified from prior literature, demonstrating that a structured, computerised feedback mechanism can be successfully embedded into an existing academic supervision workflow. Adu, Onwuegbuzie, Akomolede and Idris-Tajudeen (2024), working within a Nigerian polytechnic context similar to that of the present study, developed a web-based Final Year Project Supervision Management System specifically to address delayed feedback, poor communication and the difficulty of tracking project progress across a large student population, and reported that the resulting system enabled real-time communication and follow-up between students and supervisors regardless of

their physical location. Cavalcanti, Barbosa, Carvalho, Freitas, Tsai, Gašević and Mello (2021), in a systematic review of automatic feedback in online learning environments, found that automated feedback mechanisms consistently reduce the turnaround time between a learner submitting work and receiving a response, while also easing the workload placed on instructors who would otherwise have to provide that feedback manually. Together, these studies support the empirical expectation that replacing manual, meeting-dependent supervision with a rule-based digital platform will measurably improve both the speed of feedback and the visibility of student progress, which is precisely the outcome this study seeks to achieve at Yaba College of Technology.

2.4 Review of Related Work

This section critically reviews twenty published studies, drawn from journals and preprint archives between 2020 and 2026, relevant to digital project supervision, student progress tracking, rule-based algorithms in education, e-learning management systems, and automated academic feedback. Every source cited in this section is listed in full in the References section at the end of this document, and a PDF link is placed beside each study. For each study, the methodology, key findings and a specific limitation or gap relative to the present study are identified.

Adu, Onwuegbuzie, Akomolede and Idris-Tajudeen (2024) developed a web-based Final Year Project Supervision Management System (FYPSMS) for a Nigerian federal polytechnic, using a multi-tier architecture with PHP and MySQL to enable students to upload project drafts, update progress, and receive timely feedback from supervisors regardless of physical distance. Their methodology combined a review of the challenges facing all stakeholders students, supervisors, examiners and administrators with the design and presentation of wireframes for each user role. The system was found to bridge the time and communication gap between supervisors and students, but the study stopped short of implementing any automated, rule-based computation of a numeric progress percentage; progress was represented only as a general status indicator rather than a weighted milestone score, which is precisely the gap the present study's rule-based algorithm addresses.

Castro (2024) designed a Mobile-based Student Internship Monitoring System using a progress tracking algorithm for a Philippine college, allowing trainee students to clock in and out, log daily tasks, and share GPS-verified time records with faculty supervisors,

parents and partner agencies through an Android application. The system was evaluated by thirty stakeholders against the ISO 25010 quality standard and was rated highly for functionality, reliability and usability. However, the progress tracking algorithm in this study was built around attendance and time-logging rather than around discrete academic milestones with defined completion rules, so it is not directly transferable to the chapter-by-chapter, defence-driven structure of an HND research project, which the rule-based milestone algorithm proposed in this study is specifically designed to model.

van der Merwe, du Toit and Kruger (2020) proposed and implemented a model-based framework for computerised academic progress feedback at a South African university, identifying the elements, models and concepts necessary for an effective feedback system from prior literature before implementing and evaluating the framework through a case study and user survey. The framework was readily accepted by end users and conformed to the requirements identified in the literature, offering strong empirical support for the acceptability of computerised progress feedback among academic stakeholders. The framework nevertheless addressed feedback delivery in general terms rather than defining a specific weighted, rule-based scheme for computing a percentage-completion score from individual project milestones, a gap the present study's algorithm fill

Cavalcanti, Barbosa, Carvalho, Freitas, Tsai, Gašević and Mello (2021) conducted a systematic literature review of automatic feedback in online learning environments, synthesising findings from a large body of prior studies on how automated feedback mechanism's function and how effective they are compared with manual instructor feedback. The review found that automatic feedback consistently reduces turnaround time and helps instructors manage large cohorts, but also noted that many of the reviewed systems focus narrowly on quiz or assignment-level feedback rather than on the multi-stage, long-duration structure of a full research project, and rarely incorporate an explicit, auditable rule set of the kind proposed for milestone-based project supervision in this study.

Gorchakov, Demidova and Sovietov (2024) described a rule-based algorithm and its specialisations for measuring the complexity of software written by students in an educational digital environment, expressing established metrics such as Cyclomatic and Cognitive Complexity through the algorithm and proposing a new Educational Complexity metric for classroom use. Their methodology involved formally defining the rule-based

algorithm and validating it against a dataset of student-written programs. The study demonstrates that a rule-based approach can produce transparent, explainable scores in an educational setting, which supports the design choice of this study, but it was applied to source-code complexity rather than to project-milestone progress, and did not address supervisor-student communication or notification at all.

Angeioplastis, Konstantakis and Tsimpiris (2026) audited early-warning models built on Learning Management System log data across three institutional datasets, distinguishing predictive accuracy within familiar courses from reliability when a model is applied to an unseen course. Using several machine-learning model families evaluated under random-row, unseen-student and unseen-course validation protocols, the study found that high accuracy under conventional validation did not guarantee reliable early-warning performance once a course-level distribution shift was introduced. This finding is a caution against relying on opaque predictive models for progress or risk scoring in an academic setting, and reinforces the present study's decision to use a transparent, rule-based algorithm whose milestone weights and thresholds can be audited directly, rather than a statistical model whose behaviour on new cohorts cannot be easily verified in advance.

Zhang, Lee, Xu, Chen and Obaidat (2024) proposed an effective Learning Management System incorporating an advanced educational data mining module capable of extracting frequent and exception rules directly from stored student performance records, rather than relying on an external mining step. Their methodology combined probability-based frequent-pattern mining with iterative updates to the LMS database. The study confirms that embedding rule discovery directly within an LMS improves the efficiency of extracting actionable patterns from student data, but its focus was on mining behavioural and performance patterns from coursework rather than on tracking the discrete, sequential milestones of an extended research project such as an HND dissertation.

Chandra Mouli, Paramesh, Niharika and colleagues (2025) developed a Project Tracker Management Tool intended to track and monitor final-year academic projects, giving guides, coordinators, Heads of Department, principals and directors oversight of a project through its full lifecycle using a full-stack web application modelled loosely on industrial project-tracking tools such as JIRA. The study identifies milestone tracking as a core element that lets both students and supervisors see progress against predetermined objectives and enables early detection of problems. This work is closely aligned with the

present study in intent, but it does not specify a weighted, rule-based algorithm for computing a numeric progress percentage from milestone completion, leaving the precise computation of "how far along" a project is largely undefined — the exact gap this study's algorithm is designed to close.

Chacko, Wang, Perez-Felkner, Haiduc, Whalley and Liu (2025) presented CS-Guide, a framework that combines large language models with weekly self-reported grades and reflective journal entries to trigger tailored academic interventions for computer science students, evaluated on a four-year, approximately twenty-thousand-entry longitudinal dataset with an intervention-recommendation F1 score of up to 97 percent for first-year students. The study found that combining structured, rule-triggered interventions with unstructured qualitative data substantially improved the timeliness and consistency of academic monitoring compared with a human counsellor-to-student ratio that routinely exceeds 350 to 1. The authors also candidly noted that debugging rule-based interventions is time-intensive, since each rule update risks breaking dependencies elsewhere in the rule set a practical concern the present study addresses by keeping the milestone rule set for project supervision deliberately small and independently verifiable.

Kuzminykh, Nawaz, Shenzhang, Ghita, Raphael and Xiao (2024) developed a personalised feedback framework for online education programmes that extends a generative-AI chat model with embeddings to deliver topic-targeted feedback on quiz-based assessments, reporting efficacy rates of 90 percent and 100 percent for open-ended and multiple-choice questions respectively in a proof-of-concept evaluation. Their methodology involved building and testing the framework against a set of assessment items rather than against a full academic-project workflow. The study demonstrates the feasibility of automated, personalised feedback at scale, but is oriented toward assessment marking rather than the ongoing, milestone-based supervision of an extended piece of independent research such as an HND project.

Li, Chen, Tang, Švábenský, Deguchi, Yamashita and Shimada (2025) compared single-agent and multi-agent large language model strategies for automatically assessing open-text student reflections and predicting academic performance, using a dataset of 5,278 reflections from 377 students across three academic terms. The study found that a single-agent, few-shot prompting strategy achieved the closest match to human evaluations, and that models incorporating LLM-assessed reflection scores outperformed baseline models

at identifying at-risk students. While methodologically rigorous, the study concerns the automated scoring of free-text reflections rather than the tracking of discrete, verifiable project milestones, and does not propose any rule-based weighting scheme comparable to the one required for HND project supervision

Qian, Cheng, Guan, Dai, Jin, Yang, Nawaz, Swiecki, Chen, Yan and Gašević (2025) investigated comprehensive, automated evaluation of large language model-generated educational feedback using dedicated LLM feedback evaluators, addressing the practical problem that manually checking the quality of AI-generated feedback at scale is itself labour-intensive. Their methodology involved constructing evaluator models and comparing their judgements against human assessments of feedback quality. The study is valuable for establishing how automatically generated feedback can itself be checked for quality, but it does not address how project milestones should be defined, weighted or aggregated into an overall progress score, leaving the specific rule-based computation this study proposes unaddressed.

Adeyemi and AlOtaibi (2025) designed a feedback-driven decision support system for dynamic student intervention that uses a LightGBM-based regressor with incremental retraining, allowing educators to feed updated post-intervention results back into the model so that its predictions of student performance improve over time, and incorporating SHAP-based explanations for interpretability. The study reported a 10.7 percent reduction in prediction error after retraining. This closed-loop, feedback-driven design is conceptually close to the present study's aim of continuously updated progress computation, but the underlying mechanism is a statistical regression model rather than an explicit, human-auditable rule set, which makes it harder for a project coordinator to explain to a student exactly why their computed status changed a transparency the rule-based algorithm in this study is designed to preserve.

Abiola, Saheed and Akinola (2024) built a Student Project Management System for monitoring undergraduate projects in the Department of Computer Science at the University of Ibadan, using HTML, CSS, Bootstrap, PHP and MySQL to give students, supervisors and coordinators a centralised platform for interacting, collaborating and monitoring project progress. The study identified communication gaps and the absence of a comprehensive monitoring mechanism as major obstacles to timely feedback in the existing manual process at the case-study institution, echoing findings reported elsewhere

in the Nigerian university system. As with several of the other Nigerian case studies reviewed here, the system provides collaboration and document-management features but does not define an explicit rule-based scheme for weighting milestones or computing a numeric progress score, which remains the specific contribution of the present study.

Abubakar and Muhammed (2022) applied the Technology Acceptance Model to examine the factors predicting internet use for academic purposes among students of a Nigerian College of Education, using a survey instrument to measure perceived usefulness and perceived ease of use as predictors of actual system use. The study confirmed that both TAM constructs remained strong, statistically relevant predictors of technology adoption among Nigerian tertiary-level users. This study does not concern project supervision directly, but its findings on Nigerian students' technology-acceptance behaviour directly informed the theoretical framework adopted in Section 2.2 of this study and the emphasis placed on interface simplicity in the proposed system's design.

Hariyanto, Budiman, Yulianto and Yasin (2025) designed a web-based information system for monitoring final projects at STMIK Jayakarta, an Indonesian institution where final-project monitoring was handled manually through Google Classroom, Google Forms and WhatsApp, which made student progress difficult to track and created data-security and communication problems. Their methodology combined observation, interviews and document analysis with Unified Modeling Language (UML) modelling to specify requirements and design the system across six stages of the final-project process, from registration and guidance to defence and post-defence revision. The authors concluded that the proposed design would improve tracking, security and communication. However, the work stops at system design: no application was built or tested, and progress is followed through stage records rather than through a weighted, rule-based percentage computed from milestone completion, which the present study implements and evaluates.

Haris, Abdullah Zailani, Hasim and Abdul Rahman (2025) proposed an integrated platform for managing final year projects at the Malaysian Institute of Information Technology, Universiti Kuala Lumpur, motivated by poor communication, difficulty in monitoring progress and a lack of consolidated documentation and evaluation between students and supervisors. The platform uses web technologies to streamline coordination among supervisors, final year project coordinators and students in supervising, monitoring and evaluating progress, and the authors report that it offers a better process flow for all

stakeholders. The study is closely aligned with the present one in its stakeholder structure, but its reported contribution is a coordination and process-flow platform rather than an explicit, weighted rule set for computing a numeric progress score from milestones, which is the specific contribution of this study.

Isa, Othman, Ali, Azizan and Ferguson (2024) developed a prototype web-based Final Year Project Management System to monitor student performance in the Faculty of Computing and Multimedia at Universiti Poly-Tech Malaysia, where rising student numbers each semester had turned manual project-management procedures into a major problem and made ineffective supervision and monitoring of student progress more likely. The study positions a centralised system as the remedy and demonstrates it through a prototype, which supports the argument that manual monitoring does not scale with enrolment. As a prototype-stage study, however, it reports no weighted milestone scheme or rule-based computation of percentage completion, and its monitoring is not tied to the defence-driven stages of an HND project.

Tavares and Soares (2025) developed and evaluated a large language model-powered tool that gives students formative feedback on their biweekly progress reports in an engineering Capstone programme at Insper in Brazil, where each student's advising professor monitors progress through regular written reports. Using usage data from two rounds of reporting among 76 students, they found an initial barrier to adoption and low engagement, but students who used the tool improved the completeness and quality of their reports, and the tool's task-parsing ability also helped reveal organisational problems in student work. The study supports the value of regular, structured progress reporting, but its tool targets the quality of report writing rather than the measurement of project completion, and its reliance on a generative model makes its outputs harder to audit than the explicit rules proposed in this study.

Shakil and Denny (2024) reported the iterative development of a large computer science capstone course at the University of Auckland over six consecutive sessions as enrolment grew substantially, describing the pedagogies used to sustain student engagement and motivation, as assessed through end-of-course surveys and students' written reflections. Continuous monitoring in the course relied on weekly progress meetings and fortnightly student journals, and the authors note that coordinating a large cohort demands considerable staff time and energy. The study therefore reinforces the

coordination burden that motivates the present work, but as a pedagogical experience report it describes no software system or algorithm for tracking progress, leaving the automation of milestone monitoring unaddressed.

Summary of Reviewed Literature

S/N	Author/Year	Focus	Methodology	Key Finding	Limitation/Gap
1	Adu et al. (2024)	Digital project supervision (Nigeria)	System design; multi-tier web architecture	Bridges time/communication gap between students and supervisors	No rule-based numeric progress score
2	Castro (2024)	Mobile progress tracking (internship)	Mobile app; ISO 25010 evaluation	GPS-verified tracking rated highly usable and reliable	Attendance-based, not milestone/defence-based
3	van der Merwe et al. (2020)	Computerised progress feedback	Framework design; case study; survey	Framework readily accepted by staff and students	No weighted milestone rule scheme
4	Cavalcanti et al. (2021)	Automated feedback (systematic review)	Systematic literature review	Automatic feedback reduces turnaround time, eases workload	Focus on assignment-level, not project-level, feedback
5	Gorchakov et al. (2024)	Rule-based algorithm (code complexity)	Algorithm design; dataset validation	Rule-based scoring gives transparent, explainable metrics	Applied to code complexity, not project milestones
6	Angeioplastis et al. (2026)	LMS early-warning models	Cross-dataset ML audit	High accuracy does not guarantee reliability on unseen courses	Motivates transparent rules over opaque ML models
7	Zhang et al. (2024)	E-learning management system	Embedded rule-mining module	Improves efficiency of extracting behavioural rules from LMS data	Mines coursework patterns, not project milestones
8	Chandra Mouli et al. (2025)	Final-year project tracker	Full-stack web application	Milestone tracking aids early problem detection	No defined weighting/scoring algorithm
9	Chacko et al. (2025)	LLM academic monitoring feedback	LLM + longitudinal dataset (4 yrs)	Up to 97% F1 in intervention recommendation	Rule maintenance is time-intensive at scale

S/N	Author/Year	Focus	Methodology	Key Finding	Limitation/Gap
10	Kuzminykh et al. (2024)	Personalised e-learning feedback	Generative-AI feedback framework; proof of concept	90-100% efficacy on quiz-based feedback	Assessment-focused, not project-supervision-focused
11	Li et al. (2025)	Automated reflection assessment	Single- vs multi-agent LLM comparison	Single-agent, few-shot best matches human raters	No milestone weighting scheme proposed
12	Qian et al. (2025)	Evaluating LLM-generated feedback	LLM feedback evaluators	Enables scalable quality-checking of AI feedback	Does not address milestone definition or weighting
13	Adeyemi & AIotaibi (2025)	Feedback-driven decision support	LightGBM regressor; incremental retraining; SHAP	10.7% reduction in prediction error after retraining	Statistical model less auditable than explicit rules
14	Abiola et al. (2024)	Undergraduate project monitoring (Nigeria)	Web application (PHP/MySQL/Bootstrap)	Centralises collaboration; improves visibility	No rule-based progress percentage computation
15	Abubakar & Muhammed (2022)	TAM / technology acceptance (Nigeria)	Survey; TAM constructs	Perceived usefulness/ease of use predict adoption	Not project-supervision specific; informs UI design only
16	Hariyanto et al. (2025)	Final-project monitoring system design (Indonesia)	UML-based design; observation and interviews	Web-based design improves tracking, security and communication	Design only; no weighted progress score
17	Haris et al. (2025)	Integrated final year project platform (Malaysia)	Web platform for supervisors, coordinators and students	Streamlines supervision and progress monitoring	No explicit weighted milestone rule scheme
18	Isa et al. (2024)	Final year project management prototype (Malaysia)	Prototype web system development	Manual monitoring does not scale with enrolment	Prototype level; no rule-based progress percentage
19	Tavares & Soares (2025)	LLM feedback on capstone progress reports	LLM tool; usage study (76 students)	Engaged users improved reports; initial adoption low	Targets report writing, not milestone completion

S/N	Author/Year	Focus	Methodology	Key Finding	Limitation/Gap
20	Shakil & Denny (2024)	Large-scale capstone course coordination	Experience report; surveys and reflections	Scaling needs engagement strategies and heavy coordination	No software tracking system or algorithm

2.5 Gap in Literature

The reviewed literature identifies complementary concerns: communication and document management in project-supervision systems; clear, interpretable rules for educational monitoring; and scalable feedback or early-warning approaches. The present study addresses their intersection through a department-scoped supervision workflow in which students select supervisors, propose topics, form registered teams, and submit work for explicit academic review. The implemented progress measure uses five equally weighted chapter approval states rather than a statistical prediction. A separate health measure explains inactivity and defence-related concerns, while notifications and an activity log record the decisions behind the displayed states. This combination provides an institution-specific implementation for the Computer Science HND case study. It is an applied integration contribution; it does not establish that no other system has similar features, or that equal chapter weighting has been empirically shown to be optimal. Institutionally validated weights and a comparative user evaluation remain possible extensions.

CHAPTER THREE

SYSTEM ANALYSIS AND DESIGN

3.0 Introduction

This chapter presents the analysis and design of the Digital Project Supervision and Progress Tracking System for the Department of Computer Science, Yaba College of Technology. It translates the supervision concerns identified in Chapters One and Two into requirements, service relationships, data structures, access controls and explicit progress rules. The design descriptions correspond to the current application source rather than the superseded email-based assignment workflow.

The system supports students, supervisors and the Head of Department (HOD). A student can identify supervisors in the same department, submit a topic, include registered project partners and work in the shared project workspace. The assigned supervisor decides the topic, reviews and grades submissions, approves chapter stages, decides extensions and grants final approval. The HOD monitors projects and supervisory activity in the same department. A separate project-coordinator account type is not implemented; departmental oversight in this version is provided through the HOD role.

The chapter proceeds from the development methodology and analysis of the existing and proposed systems to requirements, architecture, data design, milestone and health algorithms, security controls and implementation tools.

3.1 System Development Methodology

A sequential system-development lifecycle provided the initial structure for the work. Requirements analysis was followed by system design, implementation, testing and deployment, consistent with a Waterfall organisation of the main deliverables. Subsequent corrections were incorporated through revisions to affected requirements, data handling and interfaces, followed by regression checks. This description refers to the development process; it does not imply that deployment prevents later maintenance or that all requirements were known without revision.

Requirements analysis identified the information and decisions needed by each role. Students require a departmental supervisor directory, a topic-review state, access for their project partners, a chapter breakdown and retrievable feedback. Supervisors require student matriculation numbers, submission review and grades, activity history, deadline control and final-approval conditions. Department administration requires project-health and

supervisory-workload summaries. Authentication, project membership and department scope must be checked because student records and submitted documents are private.

System design specified role-specific routes, trusted server mutations, Firestore project records and subcollections, private file transfer and deterministic status processing. Project creation and key submission/review changes use transactional updates so that related records and counters can be maintained together. The project-health engine is evaluated on relevant operations and through the configured daily Vercel status job. Chapter Four documents these modules, the available verification results and the live acceptance cases.

Functional verification uses acceptance cases derived from the requirements and repeatable tests of the actual route handlers with simulated service dependencies. Public-page browser checks assess layout and signed-out behaviour. These sources of evidence have different limits: simulated services do not establish production database-rule or storage readiness, and a screenshot alone does not prove persistence. Claims about reduced feedback time, adoption or improved academic outcomes require a separate user study with defined participants, baseline observations and outcome measures.

3.2 Analysis of the Existing System

The existing project-supervision process described in Chapter One is largely manual. Students prepare project documents and arrange meetings with supervisors; progress notes and correction history may be kept in personal logbooks, notebooks or separate files. Supervisors coordinate several projects while relying on direct meetings and messages to learn whether students have completed key work. Department administrators may have to request updates individually because project information is not presented in one shared system.

This arrangement creates several operational problems. Records can be incomplete or difficult to retrieve, especially when the person holding the record is unavailable. There is no consistent mechanism for showing the latest project status to all authorised parties. Students may not know whether a submission has been received or when feedback is available. A department-level view of inactivity, unresolved issues and supervisor workload is also difficult to assemble from separate records.

The manual process remains useful for face-to-face supervision, but it does not provide a dependable, timestamped digital record. The proposed system is intended to

supplement supervision by centralising project records and making communication events visible. It does not replace academic judgement or determine whether a student has met the substantive quality requirements of a project.

3.3 Analysis of the Proposed System

The proposed system is a responsive web application with role-specific dashboards. A student first registers a valid matriculation number and signs in. The system returns supervisors from the student's department, and the student selects one before entering a topic and description. Up to four registered partners can be specified by matriculation number. The server checks the accounts and assignments, creates a pending topic and links all accepted members to the same project. The supervisor receives a submission notification and each added partner receives a project-membership notification. Supervisor-initiated project creation remains available as an alternative for an existing departmental student, with the student identified by matriculation number in the interface.

The supervisor can approve a topic, decline it with feedback or leave it pending. A lead student can revise a declined topic and return it for review. An approved topic permits chapter work, and the project workspace presents five chapter states, submission versions, feedback and grades. An approved chapter contributes 20 per cent to progress; submitting a newer version returns that stage to review until the new version is approved. All five stages must be approved before final submission and final approval. Students can request deadline extensions, which the assigned supervisor may grant or decline. Final approval, chapter progress and project health remain separate values, and defence scheduling records a date without implementing a defence panel or result module.

The implementation separates presentation, trusted operations and persistent services. Next.js provides the interface and API routes. Firebase Authentication identifies users and supplies role and department claims; Firestore stores project, team and interaction records; a private Supabase Storage bucket holds document bytes; and Vercel hosts the application and scheduled health refresh. Notifications are recipient records inside the application, while project activity records preserve topic, submission, review and extension events. The HOD workspace aggregates departmental project and supervisor activity.

3.4 System Requirements

3.4.1 Functional Requirements

ID	Requirement
FR1	Allow new users to register a role and department; require a valid, unique matriculation number for students.
FR2	Support student matriculation-number/password login and account email/password login; verify role-specific access and clear client/server sessions on logout.
FR3	Show students the registered supervisors in their own department and allow supervisor selection and topic submission.
FR4	Allow the lead student to include up to four eligible registered partners by matriculation number; link each member to the shared project and notify added partners.
FR5	Allow only the assigned supervisor to approve, decline or mark a topic pending; allow the lead to revise a declined topic.
FR6	Present five chapter states and derive progress at 20 per cent per approved chapter.
FR7	Store project files privately, retain submission metadata and versions, and support chapter revisions through authorised upload and download routes.
FR8	Allow the assigned supervisor to decide submission review status, enter feedback and record a grade from 0 to 100.
FR9	Support submission-scoped comments, recipient notifications, project tickets and timestamped workflow activity.
FR10	Allow project students to request deadline extensions and the assigned supervisor to grant or decline them.
FR11	Require all five chapter stages to be approved before final submission and final approval.
FR12	Allow the supervisor to set the next deadline and defence date, and display those dates to authorised participants.
FR13	Identify students and teams by matriculation number in lecturer project views and roster.
FR14	Provide the HOD with project, supervisor and activity summaries scoped to the same department.
FR15	Evaluate project health from explicit activity and schedule rules during supported operations and the configured daily refresh.

3.4.2 Non-functional Requirements

Quality attribute	Requirement
Security	Verify authenticated sessions, roles, membership and department; use private file storage and server-validated workflow mutations.
Usability	Provide role-specific navigation, legible phone inputs, wrapping layouts and mobile navigation without page-width overflow.
Availability	Use hosted managed services and show actionable errors for unavailable reads, writes or file transfers.
Integrity	Validate matriculation identifiers, partner assignments, grades, dates and chapter/final gates; update related records atomically where implemented.
Auditability	Retain actors, timestamps, review outcomes, submission versions and decision notes in authorised project history.
Maintainability	Separate interface, typed models, server validation, storage access and deterministic progress/health logic.
Performance	Use paginated lists and stored dashboard aggregates to avoid reading every project for each summary view.

3.5 System Architecture

The architecture separates the responsive browser interface, authenticated server logic and external data services. Firebase Authentication supports email/password sign-in; student matriculation-number login is resolved and verified through a server endpoint before a Firebase client login is established. The browser synchronises the authenticated identity with an HttpOnly server session cookie. Protected layouts and API routes verify that session and apply role, department and project-membership checks. Firestore live reads are constrained by database security rules, while workflow mutations use trusted server routes.

Submission files are stored in a private Supabase Storage bucket. The browser obtains authorised signed upload or download access from a server route and transfers the file directly to storage; Firestore retains the file reference and review metadata. This avoids placing file bytes in the main database or making the bucket public. The configured daily Vercel job re-evaluates stored project-health classifications, while relevant writes also invoke the rule engine. Figure 3.1 shows the service relationships and Figure 3.2 shows the principal records. Vercel application deployment and Firebase rule/index deployment are separate operations.

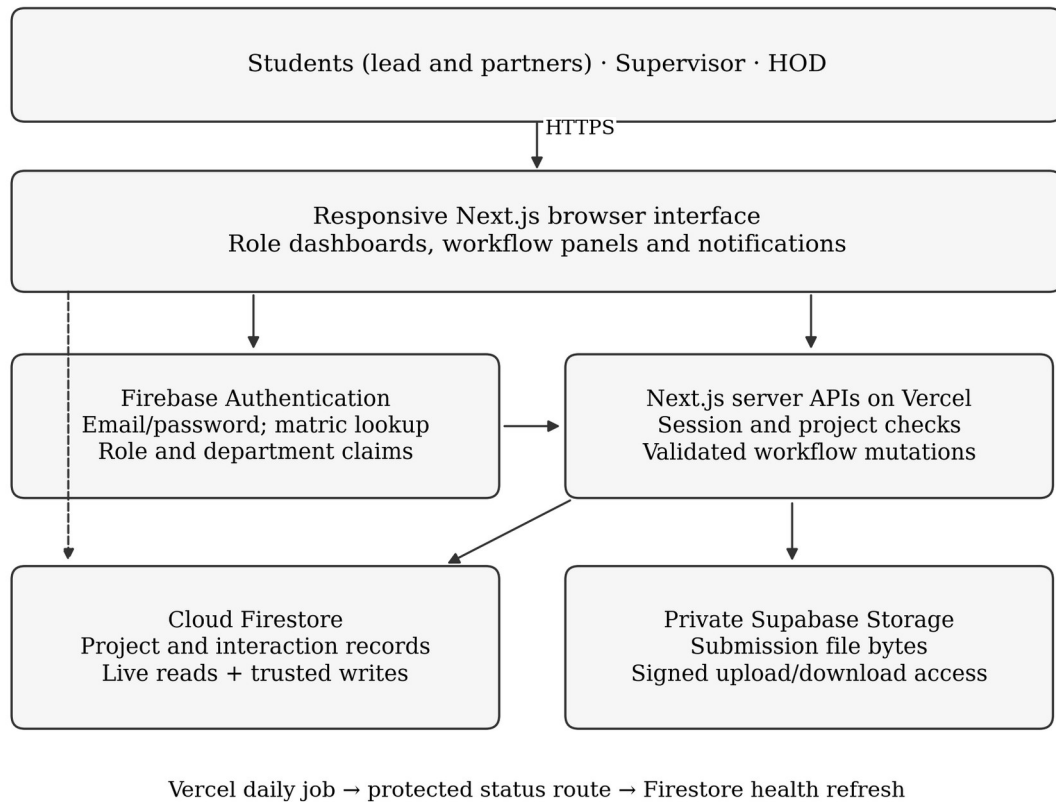


Figure 3.1: Architecture of the Digital Project Supervision and Progress Tracking System

3.6 Database and Data Design

The application uses Firestore documents and subcollections. A user record holds a role, department and, for students, a matriculation number and project link. A server-only matriculation index resolves the student's identifier to a Firebase user ID. Each project identifies its lead student, member IDs, assigned supervisor and department, and stores topic decisions, five chapter states, progress, final approval, extension details, deadlines and activity timestamps. Tickets, submissions and activity entries belong to the project; comments belong to a submission. User-scoped notifications and supervisor/department aggregates support dashboards. Arrays of names and matriculation numbers provide readable team identifiers, while user IDs remain the access-control references.

Collection / record	Main fields or purpose	Relationship
users/{uid}	Name, email, role, department, matricNumber, projectId, supervisorId	User account and assignment; staff views use student matriculation numbers.

Collection / record	Main fields or purpose	Relationship
matric_index/{key}	Normalised matriculation number and uid	Server-only student identifier lookup.
projects/{projectId}	Lead and member IDs; supervisor; topic state; five chapter states; progressPct; finalApproved; defence/deadline and extension fields	Shared project; access follows membership, assignment or department HOD.
projects/{id}/tickets/{ticketId}	Title, body, priority, dueDate, status and author	Project question or task.
projects/{id}/submissions/{submissionId}	Kind, chapterNumber, version, status, grade, reviewFeedback, storagePath and firstResponseAt	Project submission metadata; file bytes reside in private Supabase Storage.
.../submissions/{id}/comments/{commentId}	Author, role, body and timestamps	Discussion attached to one submission.
projects/{id}/activity/{activityId}	Event kind, actor, title, note/date and timestamp	Topic, submission, review and extension audit history.
users/{uid}/notifications/{id}	Kind, title, href, actor, read and createdAt	In-application notification for one recipient.
dashboard_stats/{supervisorId}	Workload, pending reviews, tickets, health counts and response metric	Supervisor aggregate; readable by that supervisor and same-department HOD.
activity_daily/{department_day}	Daily submission, comment and ticket counts	Department-scoped activity chart.

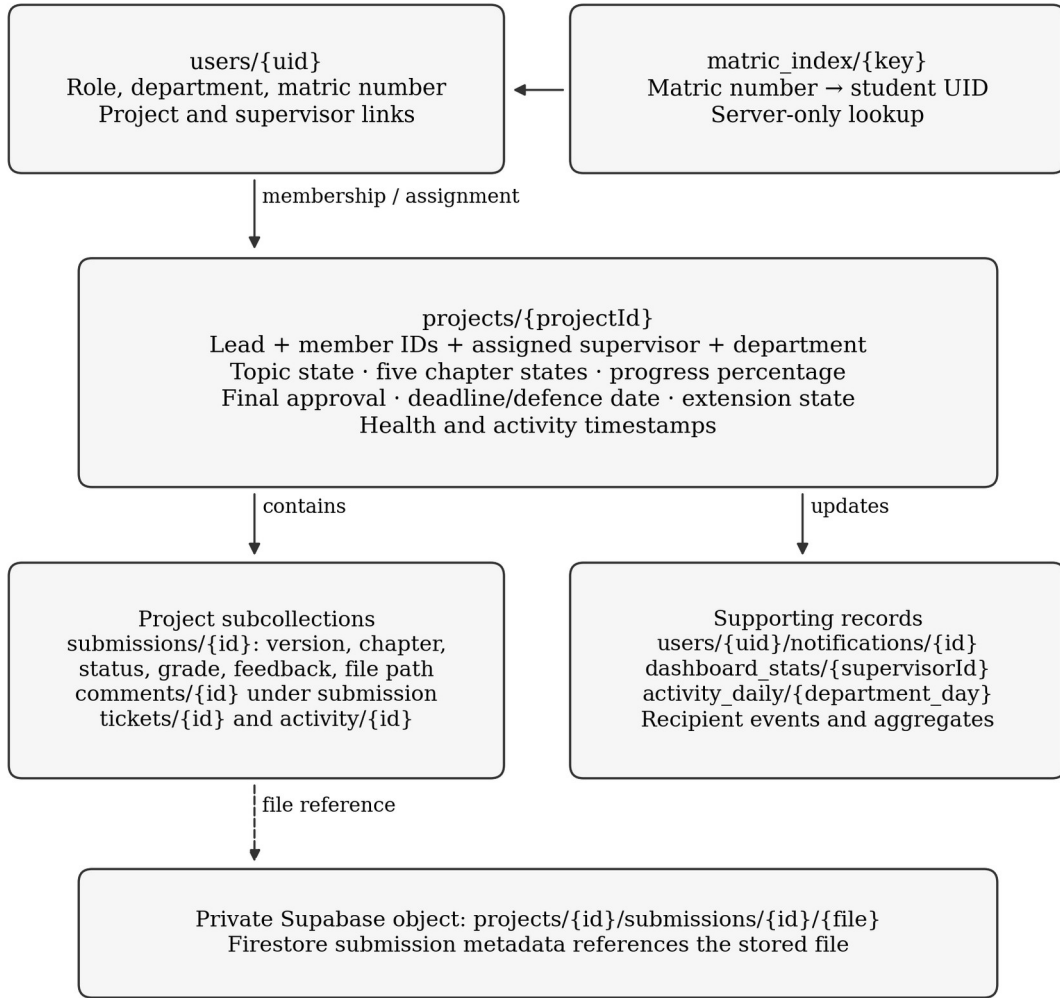


Figure 3.2: Firestore Records, Project Membership and Private Submission Storage

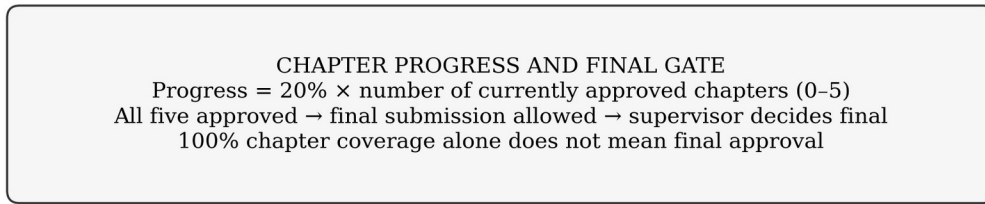
3.7 Rule-based Milestone Progress and Project Health Algorithms

The implementation maintains two related but distinct outputs: chapter-based progressPct and the categorical milestoneStatus used for project health. Let A_i equal 1 when the current state of chapter i is approved and 0 otherwise. For five chapters with equal weights $w_i = 20$, progress is $P = \sum(w_i \times A_i)$, for $i = 1, \dots, 5$. Consequently, zero, one, two, three, four and five approved chapters yield 0, 20, 40, 60, 80 and 100 per cent respectively. Pending review and requested changes contribute no weight. The server derives this value from chapter states; students cannot submit an arbitrary progress percentage.

Topic approval is a prerequisite for chapter work rather than a separately weighted milestone. A chapter or revision submission moves its stage to In review; the assigned supervisor's decision changes it to Approved or Changes requested. The latest submission reference prevents an older chapter review from overwriting the stage of a newer version. Replacing an approved chapter with a new version returns that chapter to review and can

reduce the progress percentage. All five stages being approved permits a final submission, but the supervisor must still approve that final document before `finalApproved` becomes true and the project lifecycle status becomes approved. A displayed 100 per cent therefore indicates approved chapter coverage and does not, by itself, prove final clearance or successful defence.

Project health is evaluated separately using ordered rules. No recorded activity, or inactivity greater than 21 days, yields `Stalled`. Otherwise, a defence date that has passed without final approval, or a date within the next 30 days without final approval, yields `Behind`. A further `Behind` condition applies when defence is within 45 days and no submission exists or the last submission is more than 45 days old. Finally, fewer than one ticket in the preceding 30 days combined with inactivity greater than 10 days yields `Behind`. A project that matches none of these conditions is `On Track`. The first matching rule wins, so inactivity takes priority over defence pressure.



PROJECT HEALTH: evaluate in order; first matching rule wins

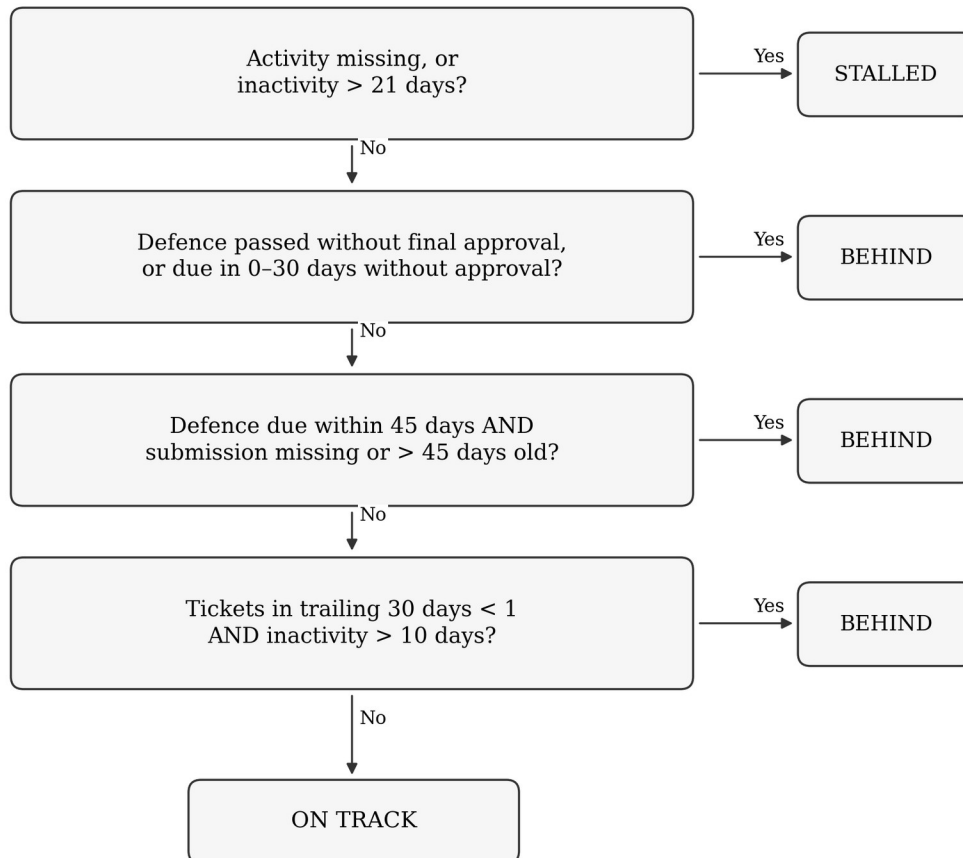


Figure 3.3: Chapter Progress, Final-Approval Gate and Ordered Project-Health Rules

Figure 3.3 summarises both computations. Health indicators are monitoring signals rather than grades: a project can have a high approved-chapter percentage and still be flagged for inactivity, and final approval does not bypass the first inactivity rule. The thresholds are fixed implementation values. Event notifications accompany the corresponding workflow actions, while the health result and its explanatory reason appear in project views and summaries; the daily refresh does not implement email or SMS reminders. Chapter Three describes the rules and Chapter Four separates automated checks from the live acceptance procedure.

3.7.1 Progress and Status Evaluation Pseudocode

Algorithm 3.1: Chapter Progress and Project Health Classification

```
FUNCTION computeProgress(chapterStates)
  approvedCount = COUNT(state = APPROVED)
  RETURN approvedCount × 20
END FUNCTION

FUNCTION classifyHealth(project, now)
  IF activityMissing OR inactivityDays > 21 THEN RETURN STALLED
  IF defenseDate exists THEN
    d = daysUntilEndOfDefenseDate(now)
    IF d < 0 AND NOT finalApproved THEN RETURN BEHIND
    IF 0 ≤ d ≤ 30 AND NOT finalApproved THEN RETURN BEHIND
    IF d ≤ 45 AND (submissionMissing OR submissionAge > 45) THEN
      RETURN BEHIND
    END IF
  IF ticketsLast30Days < 1 AND inactivityDays > 10 THEN RETURN BEHIND
  RETURN ON_TRACK
END FUNCTION

FINAL SUBMISSION AND APPROVAL REQUIRE five chapter states =
APPROVED
```

3.8 Security and Access Control

Identity, role assignment and operation authorisation are separate concerns. Firebase Authentication verifies the account, and the provisioning endpoint sets the selected role and department once for a new account. Existing provisioned accounts cannot use that endpoint to change role; protected layouts, server APIs and Firestore rules enforce the resulting permissions. Students access projects through lead or partner membership, supervisors through assignment, and HOD users through department scope. Sensitive project and submission mutations are denied to direct client database writes and are validated by server handlers.

Staff self-registration is intentionally available in this version, using distinct registered email accounts. The application does not implement a hardcoded staff-email list, an institutional email-domain restriction, invitations or independent verification of employment. Role enforcement prevents an existing student account from exercising supervisor actions, but it does not prove that a newly registered person selecting a staff role is an authorised employee. Controlled staff onboarding would require an additional institutional policy. Private storage and signed links protect submission access. Logout invalidates Firebase client sign-in, clears the HttpOnly server cookie after pending session

writes, and performs fresh navigation so that cached dashboard controls do not remain on the landing page. Credentials and private document links should not appear in report screenshots.

3.9 Development and Deployment Tools

Layer	Technology observed in the deployed implementation	Role
Application framework	Next.js 14 App Router	Web pages, routing and server API endpoints
Language	TypeScript	Typed application and domain logic
Interface	Tailwind CSS and shadcn/ui components	Responsive layouts and common controls
Authentication and database	Firebase Authentication and Cloud Firestore	User identity, role claims and structured application data
File storage	Supabase Storage private bucket	Project-submission file storage and signed access
Hosting and scheduled work	Vercel with scheduled cron route	Application deployment and daily status refresh

The implemented stack consists of Next.js 14, React, TypeScript, Tailwind CSS, shadcn/ui-style components, Firebase Authentication, Cloud Firestore, private Supabase Storage and Vercel hosting. Recharts supports dashboard charts, and Node's test runner exercises workflow and session logic. Playwright with Chromium and WebKit provides browser checks. The interface uses locally bundled Roboto and YABATECH green (#006600) and yellow (#FDC800), whereas the project report retains the corrected document's Times New Roman formatting. Daily cron processing is configured in Vercel; the repository also contains Firebase function code, which is not evidence that those functions have been deployed.

CHAPTER FOUR

SYSTEM IMPLEMENTATION, TESTING AND RESULTS

4.0 Introduction

This chapter describes the implementation of the Digital Project Supervision and Progress Tracking System and relates its behaviour to the requirements in Chapter Three. It presents the current account, project, team, chapter-review, extension and approval workflows, the data and file-transfer mechanisms, and the verification evidence available at the time of writing. The report describes the implementation reviewed on 8 October 2026; it does not use older screenshots as evidence of the revised workflow.

The chapter separates three types of evidence: functionality present in the source, results from repeatable tests, and production acceptance cases that require authenticated accounts and service configuration. This distinction prevents a passing public-page check or simulated database test from being reported as a completed live academic trial. Figures 4.1–4.8 have defined capture slots so that current screenshots can be added without changing the chapter structure.

4.1 Implementation Overview

The application is deployed as a Next.js 14 App Router application on Vercel. React components and Tailwind CSS implement the browser interface, while TypeScript models define users, projects, tickets, submissions, comments and notifications. Next.js route handlers verify server sessions and perform the trusted mutations. Firebase Authentication and Cloud Firestore manage user identity and structured records respectively. Supabase Storage holds submitted documents in a private bucket, with access obtained through signed URLs issued after a project-access check.

The public landing page introduces the system and provides registration and sign-in links. An authenticated user is directed to the dashboard associated with the recorded role. Students use a matriculation number and password or an account email and password; supervisors and HOD users use their account email and password. Student identifiers are normalised to uppercase and checked against the format F/HD/24/3211001: the first segment is F, D or P for full-time, distance-learning or part-time programmes; the second is HD or ND; the year has two digits; and the final identifier has seven digits. The matriculation number is an application login identifier mapped to a Firebase user, rather than a separate password system.

Registration provides Student, Supervisor and HOD role choices. Student matriculation numbers are unique and indexed by the server. A provisioned account cannot change its role by choosing another dashboard URL or repeating the provisioning call. Staff email accounts are distinct Firebase accounts, but there is no institutional staff-email allowlist or employment-verification step. This is the implemented registration policy and should not be described as verified staff-only onboarding.

Students without a project can view the departmental supervisor directory and submit a topic with a selected supervisor and eligible partners. The supervisor workspace provides topic decisions, project monitoring, submission review and grading. Students and partners use the same project record for chapters, feedback, tickets, extensions and history. The HOD workspace offers departmental project and supervisor summaries. Notifications are stored for their intended recipients and displayed through the interface; the project activity log retains the underlying workflow events.

4.2 User Interface Implementation

4.2.1 Student Project Setup and Matriculation Identity

The student dashboard presents the supervisors registered in the same department. A student selects a supervisor, enters a topic and project description, and optionally adds up to four project partners by matriculation number. Server validation checks supervisor role and department, partner existence, student role, department and existing project assignment. The creation transaction links the lead and accepted partners to one project, sets the topic to Pending, records the submission event and notifies the supervisor and partners. A rejected request must not be presented as successful project creation. Figure 4.1 should show the directory and the completed setup form using synthetic student identifiers.

[Figure 4.1 screenshot to be inserted: Capture the departmental supervisor directory and the filled topic form, including a partner matriculation number.]

Figure 4.1: Departmental Supervisor Selection and Student Topic/Team Setup

4.2.2 Topic Decisions and Shared Project Membership

The assigned supervisor can mark the topic Pending, Approve it or Decline it, with an optional decision note. The current state and feedback appear in the project workflow panel. A declined topic can be revised and resubmitted by the lead student, returning it to Pending and notifying the supervisor. Chapter submissions are blocked until the topic is

approved; the server permits proposal documents while a topic is awaiting a decision. Figure 4.2 should show the supervisor decision controls and the student activity entries resulting from the topic review cycle.

[Figure 4.2 screenshot to be inserted: Capture supervisor topic-decision controls and student activity showing a decline, resubmission and approval.]

Figure 4.2: Topic Review Controls and Student Decision History

A partner is a member of the actual project, not merely a recipient of a link. The partner profile stores the project association, membership authorises the shared workspace and file access, and a notification identifies the student who added the partner. The partner dashboard loads the same project, chapter progress and supervisor as the lead. Figure 4.3 should show that notification and the linked workspace; student email addresses are not the lecturer-facing project identifier.

[Figure 4.3 screenshot to be inserted: Capture the added partner's notification panel and the shared project showing the team's matriculation numbers.]

Figure 4.3: Partner Notification and Shared Project Access

4.2.3 Chapter Submission, Review, Feedback and Grading

The workflow panel lists five chapter stages: Introduction; Literature Review; Methodology; System Design and Implementation; and Summary, Conclusion and Recommendations. These are the application's stored stage labels. A stage is Not started before a file is submitted, In review after submission, Changes requested when corrections are required, and Approved when the supervisor approves its current version. The report's Chapter Three title remains System Analysis and Design and Chapter Four covers implementation, testing and results, following the corrected report structure. Chapter-number mapping, rather than exact equality of the interface labels and report titles, connects uploaded chapters to their stages.

The submission form records the document title, kind and chapter number and selects the file to upload. The client permits PDF, Word, PNG and JPEG files up to 25 MB. It first obtains a signed upload URL, sends the bytes to private storage with an upload-progress indicator, and then registers the metadata through the project submission endpoint. A visible upload percentage is not the same as completed metadata registration; the user

should confirm that the submission appears in the project list. The record retains a project-level version number, file path, submitting user, status and timestamps.

The supervisor review workspace supports Pending review, Changes requested and Approved states, written feedback, and an optional grade from 0 to 100. The first supervisor response timestamp supports responsiveness calculations. Submission-scoped comment forms allow authorised project participants to discuss that particular file, rather than attaching all corrections to one undifferentiated project thread. Review decisions notify the project students and create activity entries. Figure 4.4 should show the chapter breakdown and a saved review with a grade and meaningful feedback.

[Figure 4.4 screenshot to be inserted: Capture the stage breakdown after one chapter approval and a saved review containing feedback and a grade.]

Figure 4.4: Chapter Progress and Submission Review with Feedback and Grade

Submitting a revision creates a new record and returns the selected chapter to In review. The project retains the latest submission reference for each chapter, so an older review cannot overwrite the newer stage. Progress is computed from currently approved stages, not from the number of files uploaded. Therefore, replacing an approved stage with a new submission can reduce its counted progress until approval is restored. The server rejects invalid grades and prevents students from setting review decisions or fabricating progress.

4.2.4 Tickets, Activity and Deadline Extensions

Students and supervisors can create project tickets to record a question or task, select a priority, and use Open, In progress, Blocked or Done states. Tickets support the supervision record but are not weighted chapter milestones. The activity panel displays topic, submission, review and extension events with actors and timestamps, making the transition from Pending to Approved or Declined retrievable. Ticket and submission/comment counts also support dashboard summaries and activity charts.

A project student can request a later deadline date. Validation requires a real calendar date later than the current deadline, or later than the current time when no deadline exists, and within the implemented 180-day request horizon. A second request cannot replace an already pending request. The supervisor can grant or decline it with an optional note. Approval updates nextDeadline to the requested date; decline leaves that deadline

unchanged. An extension does not automatically move defenseDate, which is stored separately. Notifications and activity entries make the decision visible to the project members. Figure 4.5 should show the decision controls and the persisted result.

[Figure 4.5 screenshot to be inserted: Capture a pending extension with grant/decline controls and the student record after the decision.]

Figure 4.5: Deadline-Extension Request, Supervisor Decision and Activity Record

4.2.5 Final Submission, Final Approval and Defence Date

All five chapter states must be Approved before a final submission is accepted. The same condition is checked when the supervisor grants final approval. This prevents an empty or partially populated chapter array, or a direct status change, from bypassing the final gate. Approving the final submission records finalApproved = true, changes the lifecycle status to Approved, preserves 100 per cent chapter progress, adjusts relevant dashboard counters and notifies the team. Ordinary chapter changes are blocked while final approval is active. The final review can be returned to a non-approved state by the assigned supervisor before further chapter work resumes.

The supervisor can record the next deadline and a defence date. Students and the department HOD see the same stored project dates through their authorised views. The health rules use the defence date as a schedule signal; the application does not allocate examiners, conduct the defence, record a defence result or issue a certificate. Figure 4.6 should show all five approved stages, the explicit final-approval message and the scheduled defence date. It demonstrates documented project clearance and date tracking, not an unimplemented defence-result screen.

[Figure 4.6 screenshot to be inserted: Capture the completed chapter states and explicit final approval, with the project's scheduled defence date.]

Figure 4.6: Five Approved Chapters, Final Project Approval and Scheduled Defence Date

4.2.6 Supervisor and Departmental Monitoring

The supervisor dashboard summarises active projects, pending submissions, open tickets and health classifications. Project lists and the roster use student matriculation numbers, while project views display the team. The HOD dashboard and associated views present department-scoped project and supervisor summaries, a health-status distribution,

daily activity and supervisor responsiveness. The response metric is based on recorded submission and first-response timestamps; where no measured response exists, the interface should not invent a turnaround value. Figure 4.7 should show meaningful, synthetic activity data and readable chart labels.

[Figure 4.7 screenshot to be inserted: Capture the departmental dashboard with readable project-health and activity/response summaries.]

Figure 4.7: HOD Departmental Project-Health and Supervision-Activity Overview

4.2.7 Mobile Presentation and Logout

The interface uses locally bundled Roboto with YABATECH green and yellow. Responsive layouts allow controls to wrap, tables to scroll inside their containers where necessary, and phone forms to retain legible input text. The mobile landing header places branding and account actions on separate rows at narrow widths, preventing the dashboard button from overlapping the project title. These interface choices concern the application; the report retains the corrected document's Times New Roman font and academic layout.

Logout clears both forms of authentication state: Firebase client sign-in and the HttpOnly server session cookie. The session synchronisation logic prevents a pending or late authentication callback from recreating a cookie after logout. The interface then performs a fresh landing-page navigation, and an initial signed-out state also clears server cookies left by the older flow. Restored browser-cache pages are reloaded. Figure 4.8 should show the matriculation login screen and the signed-out landing header on an iPhone-sized viewport; the latter should present sign-in/registration controls without Go to dashboard.

[Figure 4.8 screenshot to be inserted: Capture the iPhone matriculation login form and the landing header immediately after logout.]

Figure 4.8: Mobile Matriculation Login and Landing Page after Logout

4.3 Test Design and Acceptance Criteria

Testing combines automated regression checks with a defined black-box acceptance exercise. The regression suite imports actual application handlers and substitutes Firebase and storage dependencies so that valid and invalid inputs can be repeated without modifying production data. Session synchronisation is tested with controlled asynchronous requests. Browser smoke checks use the rendered public landing, login and registration

pages at 320, 375, 390, 430, 768 and 1440 pixel widths. They also examine header overlap, document-width overflow, readable mobile inputs, cookie deletion and the signed-out protected-route response.

The verification record contains 34 passing regression tests with no failed or skipped tests. The workflow and validation checks account for 25 tests, and the session synchronisation checks account for nine. In the preceding deployment verification, 18 public-page checks passed against the live Vercel site in Chromium and 18 passed against a local production build in WebKit over HTTPS. Production build, lint and TypeScript checks also completed in that verification. WebKit is relevant to Safari rendering, but desktop engine checks are not a physical-iPhone usability trial. These results establish the scope of the checks performed, not a completed authenticated production project lifecycle.

Table 4.1: Verification Evidence and Its Scope

Evidence	Recorded outcome	What it establishes / limitation
Workflow/validation regression suite	25 passed; 0 failed	Actual handlers with simulated dependencies; not live Firebase/Supabase verification.
Session synchronisation regression suite	9 passed; 0 failed	Pending-write ordering, late callback protection, stale-cookie recovery and error reporting.
Live Chromium public-page checks	18 passed in deployment verification	Public pages at six widths, layout and signed-out cookie/route checks; no real-account lifecycle.
Local WebKit public-page checks over HTTPS	18 passed in deployment verification	Safari-engine layout and cookie checks on a local production build; no physical-iPhone trial.
Build, lint, type checks and Vercel	Completed in deployment verification	Compilation/static checks and successful app deployment; not proof of external service policies.
Authenticated production acceptance	To execute	All roles, file transfer, rule/index deployment, reload persistence and screenshots.
TAM/hypothesis evaluation	Not conducted	No user sample, comparative timing study or statistical significance result.

Table 4.1 records the available evidence. Table 4.2 defines the live acceptance cases, with every case marked To execute until an authenticated run records an actual result. Each run should identify the non-secret test-account label, action, expected result, actual result, screenshot or log reference, and Pass, Fail or Blocked status. Reloading or signing in again

after a successful write checks persistence. The evidence should be captured with synthetic student data and separate browser sessions for the lead, partner, supervisor and HOD.

Table 4.2: Authenticated Production Acceptance Cases

Case	Action and expected behaviour	Live result
AT01	Register students with F/D/P and ND/HD formats; malformed/duplicate matriculation numbers are rejected. Staff registration uses email without student matriculation fields.	To execute
AT02	Log in by matriculation number and by email: the same student account/project is loaded. Wrong credentials produce no session.	To execute
AT03	Open protected routes from another role or department: access is denied. Existing provisioned roles cannot be changed by repeating signup provisioning.	To execute
AT04	Select a departmental supervisor, submit a topic and add a partner: one Pending project links both students and generates the expected notifications.	To execute
AT05	Try nonexistent, duplicate, self, already-assigned or other-department partners/supervisors: invalid requests are rejected without a successful partial team.	To execute
AT06	Set Pending, Decline with a note, revise as lead, then Approve: states, activity and recipient notices persist after reload. Chapter work stays blocked before topic approval.	To execute
AT07	Sign in as the partner: see the addition notification, same project, team and authorised document access. An unrelated student cannot read it.	To execute
AT08	Upload an allowed chapter file, download it as authorised staff, request changes with feedback and grade, then submit/approve a revision: current stage and records persist.	To execute
AT09	Try a disallowed/oversized file, invalid grade and review of an older chapter version: errors are shown and current stage/final state are not incorrectly changed.	To execute
AT10	Approve chapters individually: progress is 20/40/60/80/100. A newly submitted replacement returns that stage to review and removes its counted 20 per cent until re-approved.	To execute
AT11	Post a submission comment and create/update a ticket: the authorised project views, notifications and applicable counts reflect the action.	To execute
AT12	Request an extension, decline it, then request another and approve it: only approval changes nextDeadline; malformed dates/duplicate pending requests are rejected.	To execute

Case	Action and expected behaviour	Live result
AT13	Attempt final submission or approval with fewer than five approved chapters: reject it. With all five approved, approve the final and confirm explicit approval and team notification.	To execute
AT14	Set next deadline and defence date as assigned supervisor: participants see persistent dates. Defence results and panel allocation are not expected features.	To execute
AT15	Use synthetic data to exercise health-rule boundary conditions and HOD summaries: verify rule order, explanatory reason and department filtering.	To execute
AT16	At iPhone width, check header, forms, tables, touch/keyboard operation and logout. Signed-out landing has no dashboard CTA; back/protected navigation requires login.	To execute
AT17	Inspect matching Firebase rules/indexes and private Supabase policies; verify authorised access and prohibited direct writes/downloads using QA accounts.	To execute

4.3.1 Evaluation of the Research Hypothesis

The hypothesis in Section 1.5 proposes reduced feedback turnaround time and more accurate progress reporting compared with the manual process. Functional tests do not test statistical significance or establish those comparative outcomes. A suitable subsequent study would record the elapsed time between a submission and its first supervisor response, compare manual and digital records under a defined sampling procedure, and audit displayed progress against approved chapter states. Students and supervisors could also assess perceived usefulness and ease of use using TAM-informed questions.

The evaluation must specify participant numbers, consent, observation period, baseline conditions, missing-data handling and the statistical method before reporting a significance result. No participant sample, questionnaire score, measured percentage reduction or hypothesis acceptance is supplied by the current verification record. The hypothesis remains a research proposition pending that evaluation.

4.4 Implementation and System Behaviour

4.4.1 Progress Computation and Academic Decisions

The progress calculation is deterministic: one approved chapter yields 20 per cent; two yield 40; three yield 60; four yield 80; and five yield 100. A stage awaiting review or requiring changes contributes zero. The current implementation uses equal fixed weights; it does not expose a lecturer-controlled editor for chapter weights. Topic approval, final

approval and defence scheduling remain explicit conditions or decisions outside the percentage calculation. This prevents the report from treating a progress display as a grade or evidence that the defence has been completed.

For example, when Chapters One and Two are approved and Chapter Three is In review, the correct progress is 40 per cent. A new version of Chapter Two returns it to review, so the percentage becomes 20 until its current version is approved. When all five are approved, the percentage is 100 but `finalApproved` remains false until the supervisor decides the final submission. This behaviour is consistent with the completion conditions and equal weights described in Section 3.7.

4.4.2 Project-Health Classification and Refresh

The project-health engine checks conditions in the order specified in Algorithm 3.1. Inactivity above 21 days or missing activity produces Stalled before any defence rule is evaluated. If activity remains current, an unapproved final close to or after defence can produce Behind. Missing or stale submission activity near defence and weak ticket cadence combined with more than 10 days of inactivity provide further warnings. Otherwise the result is On Track. The result includes an explanatory reason stored for project display.

The configured cron refresh runs daily and recalculates health and supporting counters from the stored records. Workflow operations also call the engine where implemented. Calendar transitions can therefore change health even when the chapter percentage has not changed. A granted extension changes the next deadline but does not replace the defence date used by the health rules. The first inactivity rule also applies to a finally approved project; an approved project can later be flagged Stalled if activity stops. Health should therefore be interpreted as a monitoring signal alongside lifecycle and approval fields.

4.4.3 Data Integrity, File Access and Notifications

The student project-creation transaction writes the project, member assignments, activity and recipient notifications together. Submission and review transactions update the selected stage, progress, review state and associated counters. Server validation restricts final approval to the assigned supervisor and checks all five chapter stages. The latest-chapter reference protects against reviewing an older version as if it were current. These controls address important failure paths, although simulated tests do not prove all possible production concurrency or service-failure scenarios.

Firestore stores the structured record and the Supabase bucket stores the file bytes. Server signing routes check project access before granting temporary transfer access. Project members, the assigned supervisor and the same-department HOD have defined read scope, while submission review and final approval belong to the assigned supervisor. Firestore rules deny direct client project and submission mutations and preserve role, department, matriculation number and assignment fields. Those rules must be deployed to the matching Firebase project independently of Vercel; repository rules are a policy specification until their live deployment is confirmed.

Notifications are written to users/{uid}/notifications and can be marked read by their owner. Activity entries are stored under the project and record workflow decisions. A notification indicates that an event was recorded for the recipient, while a successful cross-account reload is needed to demonstrate delivery and persistence in a live acceptance run. Supervisor response metrics and departmental activity charts use stored timestamps and aggregates. Deployment configuration, the required indexes and the private storage policy remain part of the production acceptance checks.

4.4.4 Discussion of Results and Remaining Validation

The available regression results support the implemented validation of matriculation formats, department-scoped supervisor discovery, team creation, topic decisions, partner access, chapter review, grades, final gates, extension decisions, activity access and session synchronisation. The public browser results support the tested layouts and signed-out cookie/navigation behaviour. The current Vercel deployment completed successfully during the preceding verification. These findings are technical verification evidence; they do not establish user adoption, educational effectiveness or a production-tested defence lifecycle.

The principal remaining acceptance work is an authenticated run against the actual Firebase and Supabase services, with current rules and indexes confirmed, real file transfer exercised, all role sessions reloaded, and the requested screenshots recorded. Physical-iPhone interaction should check touch targets, keyboard behaviour and logout navigation. Staff self-registration remains the chosen product policy rather than an independent check of employment. Future extensions may include institution-controlled staff provisioning, validated unequal milestone weights, additional notification channels, and a separate defence-results module; they are not represented as current features.

4.5 User Guide

4.5.1 Student

- i. Register with a name, account email, password, department and valid matriculation number. Use the matriculation number and password to sign in, or use the account email option.
- ii. If no project exists, review the supervisors in the department, select one, enter a topic and description, and optionally add eligible registered partners by matriculation number. Submit and confirm the topic is Pending.
- iii. Read the supervisor's topic decision and note. The lead can revise a declined topic and resubmit it. Wait for approval before submitting chapter work.
- iv. Open the shared project, select a chapter and upload an allowed file. Confirm that its metadata appears in Submissions; then read the supervisor's feedback, grade and comments. Submit a revision where changes are requested.
- v. Use the chapter breakdown to distinguish Approved work from In review or Changes requested. Use tickets for questions/tasks and the activity log for previous topic, review and extension decisions.
- vi. Request a later deadline when necessary and read the decision. After all five chapters are approved, submit the final document and await explicit final approval. Check the separately recorded defence date.
- vii. Check notifications, including a partner-addition notice if another student linked the account to a project. Log out and confirm that the landing page shows sign-in controls and that protected pages require login.

4.5.2 Supervisor

- i. Register or sign in to the Supervisor account using its email and password. Review the dashboard, assigned projects and matriculation-number roster.
- ii. Open a submitted topic and use Approve topic, Decline or Mark pending. Include a decision note where useful. An alternative project-creation form can link an existing departmental student by matriculation number.

- iii. Open each submission, obtain the authorised download, review the current version and save a review state, feedback and an optional grade from 0 to 100. Use submission comments for further discussion.
- iv. Monitor chapter stages, progress, activity and tickets. Review extension requests and grant or decline them. Set the next deadline and the defence date separately.
- v. When all five chapters are approved, review the final submission and grant final approval only if the academic requirements are satisfied. If further chapter changes are needed, return the final review to a non-approved state first.

4.5.3 Head of Department

- i. Sign in with the HOD email account and confirm that project and supervisor summaries belong to the intended department.
- ii. Review the project-health distribution, project list, supervisor workload, response metric and daily activity chart. Open authorised detail views for follow-up.
- iii. Use monitoring information to support departmental discussions; topic decisions, submission grades, extension decisions and final approval remain the assigned supervisor's actions in this version.

4.6 Chapter Summary

This chapter has documented the current implementation from account registration and student-led project setup through topic review, shared membership, chapter submission, feedback, grading, extensions and final approval. It has distinguished approved-chapter progress from project health, final clearance and defence scheduling. The verification record contains passing regression and public browser checks, while authenticated production acceptance and a participant evaluation remain explicitly separate. The figure plan provides a focused set of current screenshots to complete the report without retaining images of the superseded workflow.

REFERENCES

References cited in Chapters One–Four are listed below. The corrected document's literature sources have been retained; links identify their supplied publication or access locations.

- Abiola, O. A., Saheed, O. O., & Akinola, S. O. (2024). Student project management system for monitoring undergraduate projects (Case study of Computer Science Department, University of Ibadan). *Journal of Applied Teaching and Learning*, 11(2), 119–130. <https://www.researchgate.net/publication/386986425>
- Abubakar, H., & Muhammed, A. (2022). Technology acceptance model (TAM) as a mechanism for predicting internet use for academic purposes among students of Nigeria Certificate in Education. *International Journal of Education, Technology and Science*, 2(2), 164–180. <https://www.researchgate.net/publication/361099367>
- Adeyemi, T. O., & AlOtaibi, N. F. (2025). Designing a feedback-driven decision support system for dynamic student intervention. *arXiv*. <https://arxiv.org/pdf/2508.07107>
- Adu, M. K., Onwuegbuzie, I. U., Akomolede, K. K., & Idris-Tajudeen, R. (2024). Mitigating the challenges of final year projects supervision in Nigeria tertiary institutions for improved quality assurance. *World Journal of Advanced Engineering Technology and Sciences*, 12(2), 179–188. https://wjaets.com/sites/default/files/fulltext_pdf/WJAETS-2024-0286.pdf
- Angioplastis, A., Konstantakis, M., & Tsimpiris, A. (2026). Reliable or just accurate? A cross-dataset audit of early-warning models under course-level distribution shift. *Computers*, 15(9), Article 572. <https://www.mdpi.com/2073-431X/15/9/572/pdf>
- Castro, E. G. M. (2024). Mobile-based student internship monitoring system using progress tracking algorithm. *International Research Journal on Advanced Science Hub*, 6(8), 204–209. <https://rspsciencehub.com/index.php/journal/article/download/840/714>
- Cavalcanti, A. P., Barbosa, A., Carvalho, R., Freitas, F., Tsai, Y.-S., Gašević, D., & Mello, R. F. (2021). Automatic feedback in online learning environments: A systematic literature review. *Computers and Education: Artificial Intelligence*, 2, Article 100027. <https://doi.org/10.1016/j.caeai.2021.100027>

- Chacko, S. J., Wang, A.-I. A., Perez-Felkner, L., Haiduc, S., Whalley, D., & Liu, X. (2025). CS-Guide: Leveraging LLMs and student reflections to provide frequent, scalable academic monitoring feedback to computer science students. arXiv. <https://arxiv.org/pdf/2512.19866>
- Chandra Mouli, K., Paramesh, P. M., Niharika, T., et al. (2025). Project tracker management tool. *International Journal on Science and Technology (IJSAT)*, 16(2). <https://www.ijسات.org/papers/2025/2/4299.pdf>
- Davis, F. D. (1989). Perceived usefulness, perceived ease of use, and user acceptance of information technology. *MIS Quarterly*, 13(3), 319–340.
- Gorchakov, A. V., Demidova, L. A., & Sovietov, P. N. (2024). A rule-based algorithm and its specializations for measuring the complexity of software in educational digital environments. *Computers*, 13(3), Article 75. <https://www.mdpi.com/2073-431X/13/3/75/pdf>
- Haris, N. A., Abdullah Zailani, N., Hasim, N., & Abdul Rahman, F. (2025). An integrated approach to supervising and managing students final year projects in higher institutions. *International Journal of Computational Thinking and Data Science*, 5(1), 19–27. <https://semarakilmu.com.my/journals/index.php/CTDS/article/download/14069/12771/88164>
- Hariyanto, F., Budiman, T., Yulianto, A. B., & Yasin, V. (2025). Designing a web-based information system for monitoring final projects. *International Journal of Engineering, Science and Information Technology*, 5(2), 142–153. <https://ijesty.org/index.php/ijesty/article/download/799/423>
- Isa, R., Othman, S., Ali, A. S., Azizan, N., & Ferguson, J. (2024). Prototype development of final year project management system to monitor student's performance. *Journal of Advanced Research in Applied Sciences and Engineering Technology*, 40(1), 164–173. https://semarakilmu.com.my/journals/index.php/applied_sciences_eng_tech/article/download/3389/3934/41888

- Kuzminykh, I., Nawaz, T., Shenzhang, S., Ghita, B., Raphael, J., & Xiao, H. (2024). Personalised feedback framework for online education programmes using generative AI. arXiv. <https://arxiv.org/pdf/2410.11904>
- Li, G., Chen, L., Tang, C., Švábenský, V., Deguchi, D., Yamashita, T., & Shimada, A. (2025). Single-agent vs. multi-agent LLM strategies for automated student reflection assessment. arXiv. <https://arxiv.org/pdf/2504.05716>
- Qian, K., Cheng, Y., Guan, R., Dai, W., Jin, F., Yang, K., Nawaz, S., Swiecki, Z., Chen, G., Yan, L., & Gašević, D. (2025). Dean of LLM tutors: Exploring comprehensive and automated evaluation of LLM-generated educational feedback via LLM feedback evaluators. arXiv. <https://arxiv.org/pdf/2508.05952>
- Shakil, A., & Denny, P. (2024). Enhancing student engagement in large-scale capstone courses: An experience report. arXiv. <https://arxiv.org/pdf/2404.03120>
- Tavares, T. F., & Soares, L. P. (2025). Automated formative feedback for short-form writing: An LLM-driven approach and adoption analysis. arXiv. <https://arxiv.org/pdf/2509.22734>
- van der Merwe, A., du Toit, T., & Kruger, H. (2020). A framework for computerized progress feedback in tertiary education. *Journal of Computer Science*, 16(3), 391–401. <https://doi.org/10.3844/jcssp.2020.391.401>
- Yaba College of Technology. (n.d.). History. <https://www.yabatech.edu.ng/history.php>
- Zhang, X., Lee, V. C. S., Xu, D., Chen, J., & Obaidat, M. S. (2024). An effective learning management system for revealing student performance attributes. arXiv. <https://arxiv.org/pdf/2403.13822>